

MIDI Music Generator Project

Project Documentation & Implementation Steps

This document outlines the steps to build a MIDI music generator. Instead of dealing with raw audio files (which are heavy and complex), this project uses the `music21` library to extract notes and chords from MIDI files. We then use that data to train a Long Short-Term Memory (LSTM) neural network to predict and generate new musical sequences.

1. Environment Setup

First, we need to install the required dependencies. We are using `music21` for handling the music files and `tensorflow` for building the model.

```
pip install music21 tensorflow numpy
```

2. Implementation Guide

Step 1: Parsing the MIDI Data

MIDI files are basically sets of instructions (which note to play, when, and for how long). This script loops through a directory of MIDI files and converts them into a list of strings so the model can process them easily (e.g., "C4" for a single note, or "2.5.7" for a chord).

```

import glob
import numpy as np
from music21 import converter, instrument, note, chord

def get_notes_from_midi(folder_path):
    notes = []

    for file in glob.glob(f"{folder_path}/*.mid"):
        try:
            midi = converter.parse(file)
            print(f"Parsing {file}...")

            parts = instrument.partitionByInstrument(midi)
            if parts:
                notes_to_parse = parts.parts[0].recurse()
            else:
                notes_to_parse = midi.flat.notes

            for element in notes_to_parse:
                if isinstance(element, note.Note):
                    notes.append(str(element.pitch))
                elif isinstance(element, chord.Chord):
                    notes.append('.'.join(str(n) for n in element.normalOrder))
        except Exception as e:
            print(f"Failed to parse {file}: {e}")

    return notes

```

Step 2: Preparing Sequences for the LSTM

LSTMs expect data in a specific format. We take a sequence of 100 notes as our input feature (X) and the 101st note as our target label (Y). We also have to map the string-based notes to integers because the neural network can only compute numbers.

```

def prepare_sequences(notes, sequence_length=100):
    pitchnames = sorted(list(set(notes)))
    n_vocab = len(pitchnames)

    note_to_int = {note: num for num, note in enumerate(pitchnames)}

    network_input = []
    network_output = []

    for i in range(0, len(notes) - sequence_length):
        sequence_in = notes[i:i + sequence_length]
        sequence_out = notes[i + sequence_length]

        network_input.append([note_to_int[char] for char in sequence_in])
        network_output.append(note_to_int[sequence_out])

    n_patterns = len(network_input)
    network_input = np.reshape(network_input, (n_patterns, sequence_length, 1))
    network_input = network_input / float(n_vocab)

    from tensorflow.keras.utils import to_categorical
    network_output = to_categorical(network_output, num_classes=n_vocab)

    return network_input, network_output, pitchnames, n_vocab

```

Step 3: Building the Model

We use an LSTM because it excels at learning long-term dependencies, which is exactly what makes up a musical melody. This architecture stacks two LSTM layers and includes Dropout to help prevent the model from just memorizing the training data.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, Activation

def create_model(network_input, n_vocab):
    model = Sequential()

    model.add(LSTM(512, input_shape=(network_input.shape[1],
network_input.shape[2]),
                    return_sequences=True))
    model.add(Dropout(0.3))

    model.add(LSTM(512, return_sequences=False))
    model.add(Dropout(0.3))

    model.add(Dense(256))
    model.add(Dropout(0.3))

    model.add(Dense(n_vocab))
    model.add(Activation('softmax'))

    model.compile(loss='categorical_crossentropy', optimizer='rmsprop')
    return model

```

Step 4: Training

Pass the prepared input and target output into the model to start training. Because deep LSTMs take a lot of processing power, running this script on a machine with a dedicated GPU is highly recommended to save time.

```

# model = create_model(network_input, n_vocab)
# model.fit(network_input, network_output, epochs=100, batch_size=64)

```

Step 5: Generating Notes

To generate a new song, we select a random sequence from our dataset to act as the initial "seed". The model predicts the next note, we append it to our sequence, drop the oldest note, and repeat.

```

def generate_notes(model, network_input, pitchnames, n_vocab,
num_notes_to_generate=500):
    start = np.random.randint(0, len(network_input)-1)
    int_to_note = {num: note for num, note in enumerate(pitchnames)}
    pattern = list(network_input[start])
    prediction_output = []

    for note_index in range(num_notes_to_generate):
        prediction_input = np.reshape(pattern, (1, len(pattern), 1))
        prediction = model.predict(prediction_input, verbose=0)
        index = np.argmax(prediction)

        result = int_to_note[index]
        prediction_output.append(result)

        pattern.append(index / float(n_vocab))
        pattern = pattern[1:]

    return prediction_output

```

Step 6: Converting Output Back to MIDI

Finally, we need to convert the array of string predictions back into `music21` Note and Chord objects so we can export it as a playable `.mid` file.

```

from music21 import stream

def create_midi_from_sequence(prediction_output,
output_filename="generated_music.mid"):
    offset = 0
    output_notes = []

    for pattern in prediction_output:
        if ('.' in pattern) or pattern.isdigit():
            notes_in_chord = pattern.split('.')
            notes = []
            for current_note in notes_in_chord:
                new_note = note.Note(int(current_note))
                new_note.storedInstrument = instrument.Piano()
                notes.append(new_note)
            new_chord = chord.Chord(notes)
            new_chord.offset = offset
            output_notes.append(new_chord)
        else:
            new_note = note.Note(pattern)
            new_note.offset = offset
            new_note.storedInstrument = instrument.Piano()
            output_notes.append(new_note)

    offset += 0.5

    midi_stream = stream.Stream(output_notes)
    midi_stream.write('midi', fp=output_filename)
    print(f"Saved generated music to {output_filename}")

```

3. Dataset Collection

A solid dataset is required to get good results. Single-instrument classical music works best for finding clear patterns. Good sources for this include the Chopin MIDI Archive or the Lakh MIDI Dataset.

4. Future Improvements

- **Rests:** The current script plays notes continuously. Updating the parser to catch empty spaces (rests) would make the output sound much more natural.
- **Dynamic Timing:** The code currently hardcodes the timing (using a fixed 0.5 offset). Extracting the actual duration of each note during the parsing phase would allow the model to learn rhythm and pacing.